

MODULE 9

Defensive RxJS and live sync

Flattening operators · rage-click defense · SignalR into EnrollmentStore

TMS client

```
submitClick$.pipe(  
  exhaustMap((d) => this.api.post(d)),  
  takeUntilDestroyed()  
).subscribe(/* once */);
```

One click must not mean four POSTs

Session goals

By the end of this session, you will:

- Tell the double-charge story UI chaos vs API order (spine golden rule)
- Replace naked `.subscribe()` on mutations with a guarded stream (Subject + flattening operator)
- Choose `switchMap` vs `exhaustMap` vs `concatMap` for TMS search, pay, and store sync (spine table)
- Build an `exhaustMap` + `takeUntilDestroyed()` payment pipeline (first encounter)
- Wire `@microsoft/signalr` hub events into `patchState` on `EnrollmentStore` (spine)
- Avoid Session 3 traps: `switchMap` on pay, leaky subscriptions, `SignalR` vs zone pitfalls (mistakes table)

The double-charge scenario

Imagine Liya (Student) paying for a course on a slow cellular link. She taps Pay the button freezes. She taps three more times in frustration.

If each tap calls `.subscribe()` straight on the HTTP POST, four identical payloads can hit the database. Liya is billed four times.

Golden rule: defend the API from the UI. The UI is chaotic; the API expects order (spine).

Breaking the habit: manual subscribing

Anti-pattern

```
// DANGEROUS for mutations (spine)

onPayClicked() {
  this.api.postPayment(data).subscribe();
}
```

- Each click spawns a parallel HTTP universe the browser runs them all at once (spine)
- Defense: funnel clicks into a stream (Subject) and put a flattening operator before the API (spine)

The RxJS door guards

Operator	How it resolves collisions	When to use
switchMap	Cancels the old request, starts the new	Search typeaheads
exhaustMap	Ignores new emissions while the inner work runs	Submit / Pay buttons
concatMap	Strict queue sequence matters	Store syncs, bulk uploads

Interview chant (spine)

switchMap cancels the old. exhaustMap drops the new. concatMap waits in line say it once, then draw the rage-click timeline from the spine gantt.

switchMap the canceller

- Use when only the latest result matters typeahead: search for "Ab" makes the "A" request garbage (spine)
- switchMap cancels in-flight HTTP on the client the server may still finish work you thought you aborted

Danger (spine)

Never use switchMap for Pay you can cancel the browser connection while the server still processes the bill.

exhaustMap the ignorer

Use when you want to block SPAM. `exhaustMap` ignores every new click while the inner HTTP request is still active.

That is the defense pattern for Submit and Pay the UI feels frozen on purpose until the first response lands (spine).

- Classic senior interview distinction pair with `switchMap` slide mantra (spine speaker note)
- CoTBE default: TMS enrollment fees, grade submit, any idempotent-but-deadly double POST

Applying exhaustMap first encounter

Pattern (spine trimmed)

```
private submitClick$ = new Subject<Payload>();

constructor() {

  this.submitClick$.pipe(

    exhaustMap((data) => this.api.post(data)),

    takeUntilDestroyed()

  ).subscribe((response) => console.log("Only fires once!", response));

}

onButtonClick(data: Payload) {

  this.submitClick$.next(data);

}
```

- Never subscribe inside the click handler push into Subject.next (spine)
- 'Subject' manual bus: .next(data) pushes clicks into one stream
- 'takeUntilDestroyed()' unsubscribes when the component leaves the DOM (spine)

Live sync with SignalR

Hub client (spine illustrative URL)

```
private hubConnection = new HubConnectionBuilder()
    .withUrl("/hubs/enrollments")
    .build();

connect() {
    this.hubConnection.on("StatusUpdated", (id: string, status: string) => {
        // Magically update the SignalStore (spine)
        patchState(this.store, updateEntity({ id, changes: { status } }));
    });

    this.hubConnection.start();
}
```

- Modern web is real-time instructors should not refresh to see new submissions (spine)
- WebSocket-style hub from [@microsoft/signalr](https://github.com/microsoft/signalr) patchState on the central store repaints every bound component (spine reactivity)

Common mistakes Session 3

Mistake	Symptom	Fix
switchMap on mutating submit	Payment succeeds on backend but client aborted	ALWAYS use exhaustMap for UI mutation buttons (spine)
Memory leaks	Application gets slower and slower over time (spine)	Add takeUntilDestroyed() to every open subscription pipeline (spine)
SignalR outside zones	WebSocket works, data arrives, but UI does not update (spine)	patchState / signals bypass old NgZone issues avoid legacy class fields (spine)

Internationalization (i18n) overview

Template markup

```
<!-- Tag text with the i18n attribute -->
```

```
<h1 i18n="@@tms-dashboard-title">Command Center</h1>
```

- Build-time pipeline: ng extract-i18n produces messages.xlf hand to translator build per locale
- Angular compiles separate physical bundles per language maximum speed, but separate deployment folders (/en/ vs /am/)
- Awareness only hands-on in Lab Session 3 Extension 3 (optional)

Hydration and SSR (awareness)

Default setup

```
// app.config.ts Angular 22
```

```
provideClientHydration()
```

- SSR pre-renders HTML on the server users see content before JavaScript loads
- Hydration attaches event listeners to existing DOM no re-render flash
- Event Replay is default since Angular 19 clicks during the JS dead zone are recorded and replayed after hydration
- Lifecycle: server render static paint JS download hydrate replay queued events

Lab brief Exercises 4 and 5

- Open Lab Session 3 handout (Exercises 4 and 5) for step-by-step instructions
- Exercise 4: RxJS defense exhaustMap on pay / grade submit
- Exercise 5: Live sync with SignalR hub listener updates EnrollmentStore
- Checkpoint: rage-click Pay fires one POST; hub event flips row without refresh
- Self-paced extensions (finish early?): Hydration Playground, Snackbar Feedback, i18n Extraction Test

Session 3 summary

- Flattening operators are door guards pick the right guard per TMS interaction
- Subject + exhaustMap + takeUntilDestroyed production pattern for pay flows
- SignalR + patchState server push becomes shared UI truth without refresh spam
- i18n build-time localization pipeline (extract translate build per locale)
- Hydration/SSR server-rendered HTML + Event Replay prevents dead clicks

What comes next

M10 Full-Stack Integration connect this Angular client to the .NET API under real browser security: CORS, HttpOnly cookies, XSRF, functional interceptors, route guards, and optimistic UI rollback.

CORS, HttpOnly cookies, XSRF, functional interceptors, route guards, and optimistic UI rollback connect Angular to .NET for real.